

Guía Práctica de Linux (Debian) — Bases para DevOps

Preparado para Matías González Ferreyra

Agosto 2026

Contents

Introducción	2
Herramientas que vas a necesitar	2
Etapla 1: Sistema de archivos y navegación	2
Teoría	2
Ejemplos	3
Tarea / Proyecto: “Organizador de archivos por script”	3
Etapla 2: Permisos y usuarios	4
Teoría	4
Ejemplos	4
Tarea / Proyecto: “Simulación de permisos de servidor”	4
Etapla 3: Gestión de paquetes con APT	4
Teoría	4
Ejemplos	5
Tarea / Proyecto: “Instalación y configuración de un servicio”	5
Etapla 4: Procesos y servicios (systemd)	5
Teoría	5
Ejemplos	6
Tarea / Proyecto: “Panel de control de servicios”	6
Etapla 5: Redes básicas	6
Teoría	6
Ejemplos	7
Tarea / Proyecto: “Diagnóstico de red de tu servidor”	7
Etapla 6: Scripting en Bash	7
Teoría	7
Ejemplos	7
Tarea / Proyecto: “Script de respaldo automático”	8
Etapla 7: Logs, discos y monitoreo básico	8
Teoría	8
Ejemplos	8
Tarea / Proyecto: “Reporte de salud del servidor”	9
Proyecto final integrador: “Servidor Debian configurado desde cero”	9

Recursos recomendados	9
Cómo subir cada proyecto a GitHub	9

Introducción

Linux es el sistema operativo detrás de la gran mayoría de servidores del mundo, y es la base absoluta de cualquier carrera en DevOps. Esta guía usa **Debian** como referencia (el sistema que ya usas), pero casi todo aplica igual a Ubuntu y otras distribuciones basadas en Debian, ya que comparten el mismo gestor de paquetes (APT) y estructura de archivos.

Cada etapa tiene:

1. **Teoría** explicada con palabras simples.
2. **Ejemplos** de comandos listos para ejecutar.
3. **Tarea o mini-proyecto**, documentado en un repositorio de GitHub, para que sirva como evidencia de práctica ante un reclutador.

Herramientas que vas a necesitar

Herramienta	Para qué sirve	Notas
Debian (ya la tienes)	Sistema operativo base	Puedes practicar en tu instalación actual o en una máquina virtual aparte para no arriesgar tu sistema Ya viene incluida
Terminal	Donde escribes todos los comandos	
VirtualBox o VMware	Crear máquinas virtuales para practicar sin miedo a romper nada	virtualbox.org (gratis)
htop	Monitor de procesos visual, más amigable que top	<code>sudo apt install htop</code>
tmux o screen	Mantener sesiones de terminal activas aunque se corte la conexión	<code>sudo apt install tmux</code>
Un editor de terminal: nano o vim	Editar archivos de configuración desde la terminal	nano viene incluido; vim se instala con <code>sudo apt install vim</code>

Recomendación importante: crea una máquina virtual con Debian limpio (usando Virtual-Box) para practicar los comandos más “peligrosos” (borrar archivos, cambiar permisos, reiniciar servicios) sin riesgo de romper tu sistema principal.

Etapa 1: Sistema de archivos y navegación

Teoría

En Linux, todo se organiza en una sola estructura de carpetas que empieza en la raíz / (a diferencia de Windows, que usa letras como C:). Algunas carpetas importantes:

- /home — carpetas personales de cada usuario.
- /etc — archivos de configuración del sistema.
- /var — datos que cambian con el tiempo, como logs.
- /bin y /usr/bin — programas ejecutables.
- /tmp — archivos temporales.

Para moverte y manipular archivos, usas comandos en vez de hacer clic:

- pwd — muestra en qué carpeta estás ahora (*print working directory*).
- ls — lista el contenido de una carpeta (ls -la muestra todo, incluidos archivos ocultos, con detalles).
- cd — cambia de carpeta (*change directory*).
- mkdir — crea una carpeta.
- touch — crea un archivo vacío.
- cp — copia archivos.
- mv — mueve o renombra archivos.
- rm — elimina archivos (rm -r para carpetas completas). **Cuidado: no hay papelera de reciclaje.**
- find — busca archivos por nombre, tipo, o fecha.

Piensa en la terminal como estar dentro del explorador de archivos, pero escribiendo instrucciones exactas en vez de hacer clic — es más rápido y automatizable una vez que lo dominas.

Ejemplos

```
pwd                # muestra dónde estás
ls -la            # lista todo, con detalles y ocultos
cd /var/log        # entra a la carpeta de logs
mkdir ~/practica_linux # crea una carpeta en tu home
touch notas.txt    # crea un archivo vacío
cp notas.txt notas_copia.txt # copia el archivo
mv notas_copia.txt respaldo.txt # renombra el archivo
rm respaldo.txt    # elimina el archivo (sin confirmación)

# Buscar archivos
find /home -name "*.txt" # busca todos los .txt dentro de /home
find / -type d -name "log" # busca carpetas llamadas "log" en todo el sistema
```

Tarea / Proyecto: “Organizador de archivos por script”

Crea una carpeta practica_linux con subcarpetas documentos, imagenes, scripts. Dentro, crea varios archivos de prueba vacíos con touch (ej. informe.txt, foto.png, deploy.sh). Luego escribe un archivo comandos_etapa1.md documentando, paso a paso con los comandos exactos que usaste, cómo:

1. Verificaste en qué carpeta estabas antes de empezar.
2. Creaste toda la estructura de carpetas.
3. Moviste cada archivo a la carpeta correspondiente según su tipo.
4. Buscaste todos los archivos .txt dentro de tu estructura.

Sube este documento a un repositorio nuevo en GitHub llamado linux-devops-practica.

Etapa 2: Permisos y usuarios

Teoría

Linux es un sistema multiusuario: cada archivo tiene un dueño y permisos que dicen quién puede leer, escribir o ejecutar ese archivo.

- Los permisos se representan como rwx (read, write, execute) para tres grupos: el **dueño**, el **grupo**, y **otros**.
- `chmod` — cambia los permisos de un archivo.
- `chown` — cambia el dueño de un archivo.
- `sudo` — ejecuta un comando con permisos de administrador (superusuario/root). Es como pedir “permiso especial” para una acción puntual.
- `whoami` — muestra qué usuario eres actualmente.
- `useradd / adduser` — crea un nuevo usuario.
- `usermod -aG grupo usuario` — agrega un usuario a un grupo (ej. el grupo `sudo` para darle permisos de administrador).

Piensa en los permisos como las llaves de una oficina: el dueño tiene su propia llave, el equipo (grupo) tiene otra llave que abre ciertas puertas, y cualquier otra persona (otros) solo puede entrar si tú decides dejarle una copia.

Ejemplos

```
ls -l archivo.txt           # muestra los permisos actuales, ej: -rw-r--r--
chmod 644 archivo.txt       # dueño: lee/escribe, grupo y otros: solo leen
chmod +x script.sh          # da permiso de ejecución al dueño
chmod 755 script.sh         # dueño: todo, grupo y otros: leer y ejecutar

sudo chown matias:matias archivo.txt # cambia el dueño y grupo del archivo

whoami                      # muestra tu usuario actual
sudo adduser nuevo_usuario  # crea un usuario nuevo
sudo usermod -aG sudo nuevo_usuario # le da permisos de administrador
groups matias               # muestra a qué grupos pertenece el usuario
```

Tarea / Proyecto: “Simulación de permisos de servidor”

En tu máquina virtual (recomendado, para no afectar tu sistema principal):

1. Crea un usuario nuevo llamado desarrollador.
2. Crea un script `deploy.sh` que solo el dueño pueda ejecutar, pero que todos puedan leer.
3. Documenta en `comandos_etapa2.md` los comandos usados y explica, con tus palabras, qué significa cada número de `chmod` que usaste (ej. por qué 755 y no 777).

Sube el documento a tu repositorio.

Etapa 3: Gestión de paquetes con APT

Teoría

Debian usa **APT** (Advanced Package Tool) para instalar, actualizar y eliminar software, de forma parecida a como una tienda de aplicaciones instala programas, pero desde la terminal.

- `sudo apt update` — actualiza la lista de paquetes disponibles (no instala nada, solo revisa qué hay nuevo).
- `sudo apt upgrade` — actualiza los programas ya instalados a su versión más reciente.
- `sudo apt install nombre_paquete` — instala un programa nuevo.
- `sudo apt remove nombre_paquete` — desinstala un programa (dejando su configuración).
- `sudo apt purge nombre_paquete` — desinstala un programa y borra también su configuración.
- `apt search palabra` — busca paquetes disponibles.
- `dpkg -l` — lista todos los paquetes instalados en el sistema.

Siempre corre `apt update` antes de instalar algo nuevo, para asegurarte de tener la lista más reciente de versiones disponibles.

Ejemplos

```

sudo apt update           # actualiza la lista de paquetes
sudo apt upgrade -y       # actualiza todo lo instalado
sudo apt install nginx    # instala el servidor web nginx
sudo apt search "servidor ssh" # busca paquetes relacionados con SSH
sudo apt remove nginx     # desinstala nginx
dpkg -l | grep nginx      # verifica si nginx sigue instalado

```

Tarea / Proyecto: “Instalación y configuración de un servicio”

En tu máquina virtual:

1. Instala el servidor web nginx con APT.
2. Verifica que está corriendo (lo verás en la Etapa 4, con `systemctl status nginx`).
3. Documenta en `comandos_etapa3.md` el proceso completo: cómo buscaste el paquete, lo instalaste, y cómo confirmarías que se instaló correctamente sin usar `systemctl` (pista: `dpkg -l`).

--- Etapa 4: Procesos y servicios (systemd)

Teoría

Un **proceso** es un programa en ejecución. Un **servicio** es un proceso que corre en segundo plano, generalmente iniciado automáticamente al arrancar el sistema (como un servidor web).

Debian usa **systemd** para gestionar servicios:

- `systemctl status nombre_servicio` — muestra si un servicio está corriendo.
- `systemctl start / stop / restart nombre_servicio` — inicia, detiene o reinicia un servicio.
- `systemctl enable / disable nombre_servicio` — hace que un servicio arranque (o no) automáticamente al encender el sistema.
- `ps aux` — lista todos los procesos corriendo actualmente.
- `htop` — versión visual e interactiva de `top`, para ver uso de CPU y memoria en tiempo real.
- `kill -9 PID` — termina un proceso a la fuerza, usando su número de identificación (PID).
- `journalctl -u nombre_servicio` — muestra los logs (registro de eventos) de un servicio específico.

Piensa en `systemctl` como el panel de control de todos los “empleados automáticos” (servicios) de tu servidor: puedes ver quién está trabajando, despedirlo temporalmente, o decirle que se presente automáticamente cada mañana.

Ejemplos

```
sudo systemctl status nginx      # ver si nginx está activo
sudo systemctl start nginx       # iniciar nginx
sudo systemctl stop nginx        # detener nginx
sudo systemctl restart nginx     # reiniciar nginx
sudo systemctl enable nginx      # que inicie automáticamente al encender

ps aux | grep nginx             # buscar el proceso de nginx
htop                            # abrir monitor visual (presiona q para salir)

sudo journalctl -u nginx --since "1 hour ago" # logs de la última hora
sudo journalctl -u nginx -f      # seguir los logs en tiempo real
```

Tarea / Proyecto: “Panel de control de servicios”

Usando el `nginx` que instalaste en la Etapa 3:

1. Inicialo, verifica su estado, y confirma en el navegador (o con `curl localhost`) que responde.
2. Detén el servicio y confirma que ya no responde.
3. Configúralo para que inicie automáticamente al arrancar el sistema.
4. Revisa sus logs de las últimas horas con `journalctl`.

Documenta cada paso con capturas o el texto de la terminal en `comandos_etapa4.md`.

Etapa 5: Redes básicas

Teoría

DevOps vive conectando servidores entre sí, así que entender redes básicas es esencial:

- `ip a` — muestra las interfaces de red y direcciones IP de tu máquina.
- `ping host` — verifica si un servidor responde y mide la latencia.
- `curl url` — hace una petición HTTP a una dirección, útil para probar si un servicio web responde.
- `ss -tuln` (o el más antiguo `netstat -tuln`) — muestra qué puertos están abiertos y escuchando conexiones.
- `/etc/hosts` — archivo donde puedes mapear nombres a direcciones IP manualmente, sin depender de DNS.
- `ufw` — firewall simple de Debian/Ubuntu, para permitir o bloquear puertos.

Piensa en un **puerto** como el número de una oficina dentro de un edificio (el servidor): la IP te lleva al edificio, el puerto te dice a qué oficina específica llamas (80 para web sin cifrar, 443 para web cifrada, 22 para SSH).

Ejemplos

```
ip a                # ver tus direcciones IP
ping google.com     # probar conectividad
curl -I localhost    # ver encabezados de respuesta HTTP de tu servidor local
ss -tuln            # ver puertos abiertos

sudo ufw allow 22/tcp # permitir tráfico SSH
sudo ufw allow 80/tcp # permitir tráfico web
sudo ufw enable       # activar el firewall
sudo ufw status       # ver reglas activas
```

Tarea / Proyecto: “Diagnóstico de red de tu servidor”

En tu máquina virtual con nginx corriendo:

1. Verifica en qué puerto está escuchando nginx (ss -tuln).
2. Confirma con curl que responde localmente.
3. Configura ufw para permitir solo los puertos 22 (SSH) y 80 (web), bloqueando todo lo demás.
4. Documenta en comandos_etapa5.md qué puertos abriste y por qué esos y no otros.

Etapas 6: Scripting en Bash

Teoría

Un **script de Bash** es un archivo con una secuencia de comandos que se ejecutan automáticamente, en vez de escribirlos uno por uno. Es la forma en que DevOps automatiza tareas repetitivas.

- Un script empieza con `#!/bin/bash` (la línea “shebang”, que le dice al sistema qué intérprete usar).
- Variables: `nombre="Matías"` (sin espacios alrededor del =).
- Condicionales: `if [ condicion ]; then ... fi`
- Bucles: `for archivo in *.txt; do ... done`
- **cron** es un programador de tareas que ejecuta scripts automáticamente en horarios definidos (ej. cada noche a las 2 AM).

Ejemplos

```
#!/bin/bash
# backup.sh - respalda una carpeta con fecha en el nombre

fecha=$(date +%Y-%m-%d)
origen="/home/matias/documentos"
destino="/home/matias/respaldos/respaldo_`fecha`.tar.gz"

if [ -d "$origen" ]; then
    tar -czf "$destino" "$origen"
    echo "Respaldo creado: $destino"
else
```

```

    echo "Error: la carpeta $origen no existe"
fi

# Dar permisos de ejecución y correr el script
chmod +x backup.sh
./backup.sh

# Programarlo con cron para que corra todos los días a las 2 AM
crontab -e
# Agregar esta línea dentro del editor de cron:
# 0 2 * * * /home/matias/backup.sh

```

Tarea / Proyecto: “Script de respaldo automático”

Crea un script `backup.sh` que:

1. Comprima una carpeta de práctica en un archivo `.tar.gz` con la fecha en el nombre.
2. Verifique que la carpeta origen existe antes de intentar respaldarla (manejo básico de errores).
3. Muestre un mensaje claro de éxito o error.

Programalo con cron para que corra automáticamente, y documenta en `comandos_etapa6.md` cómo lo configuraste y verificaste que funciona (revisa `/var/log/syslog` o `journalctl` para confirmar que cron lo ejecutó).

Etapa 7: Logs, discos y monitoreo básico

Teoría

Un servidor en producción necesita ser monitoreado: espacio en disco, memoria, y registros de eventos (logs) para diagnosticar problemas.

- `df -h` — muestra el espacio usado y disponible en cada disco/partición.
- `du -sh carpeta` — muestra cuánto espacio ocupa una carpeta específica.
- `free -h` — muestra memoria RAM usada y disponible.
- `/var/log/` — carpeta donde viven la mayoría de logs del sistema.
- `tail -f archivo.log` — muestra las últimas líneas de un archivo y se queda “siguiendo” nuevas líneas en tiempo real (muy usado para depurar en vivo).
- `grep "palabra" archivo.log` — busca líneas que contengan una palabra específica dentro de un log.

Ejemplos

```

df -h                                # espacio en disco de todas las particiones
du -sh /var/log                      # cuánto ocupa la carpeta de logs
free -h                             # memoria RAM disponible

tail -f /var/log/syslog              # seguir el log del sistema en tiempo real
grep "error" /var/log/syslog         # buscar líneas con "error"
grep -i "failed" /var/log/auth.log   # buscar intentos fallidos de acceso (sin importar mayúsculas)

```


Tarea / Proyecto: “Reporte de salud del servidor”

Crea un script `reporte_salud.sh` que genere un archivo de texto con:

1. Espacio en disco disponible.
2. Memoria RAM disponible.
3. Los 5 procesos que más CPU están consumiendo (pista: `ps aux --sort=-%cpu | head -6`).
4. Cualquier línea con la palabra “error” en el log del sistema de las últimas 24 horas.

Este script simula exactamente lo que un ingeniero DevOps revisa a diario. Documenta el resultado en tu repositorio.

Proyecto final integrador: “Servidor Debian configurado desde cero”

Usando una máquina virtual limpia de Debian, documenta paso a paso (en un solo `README.md` extenso, como si fuera una bitácora de trabajo):

1. Configuración inicial: usuario no-root con permisos sudo, actualización del sistema.
2. Instalación y configuración de un servicio web (nginx o Apache).
3. Firewall configurado con ufw, permitiendo solo los puertos necesarios.
4. Un script de automatización propio (puede ser el de respaldo o el de salud del servidor) programado con cron.
5. Explicación de cómo revisarías logs si el servicio dejara de responder.

Este proyecto es el que debes poder explicar de principio a fin en una entrevista de DevOps junior — muestra que entiendes el ciclo completo de configurar y mantener un servidor, no solo comandos sueltos.

Recursos recomendados

- **Documentación oficial de Debian:** wiki.debian.org
- **Práctica interactiva gratuita:** OverTheWire “Bandit” (overthewire.org/wargames/bandit) — un juego de terminal que enseña Linux practicando en un entorno real.
- **Siguiente paso natural:** una vez domines esto, sigue con la guía de SSH (para administrar este mismo servidor de forma remota) y luego Docker.

Cómo subir cada proyecto a GitHub

```
cd carpeta_del_proyecto
git init
git add .
git commit -m "Agrego documentación de la etapa X"
git branch -M main
git remote add origin https://github.com/MatiasGonzalez-Dev/linux-devops-practica.git
git push -u origin main
```